



QwenStudio

Running an open image model entirely on your own machine:
what it takes, what it gives you, and where it is worse.

This is an exploration, not a product.

Not an official Superside tool, not affiliated with or endorsed by Superside,
and not endorsed by the Qwen team or Alibaba. Qwen-Image 2.1 is published under
the Qwen Research License: personal and research use only.



What this is

An image generation and editing app that runs Qwen-Image 2.1 on one machine, with nothing in between. No ComfyUI, no cloud, no account, no API key. You clone it, double-click an installer, and 31 GB of weights come down once.

The installer downloads its own Python. It touches no system Python, installs nothing globally, and changes no PATH. Delete the folder and every trace is gone. Before installing anything it reads the hardware and picks a profile from it — which PyTorch wheel, which dtype, whether to quantise, how much to offload, what resolution to cap at — so a 10 GB card and a 48 GB card both get something that runs.

Installing it

Step	Windows	macOS
1. Get the folder	clone it, or unzip QwenStudio.zip	the same
2. Install	double-click INSTALL.bat	run ./install.command
3. Start it	double-click RUN.bat	run ./run.command

The installer builds a private .venv with uv, probes the machine and writes config.json from what it finds. The first start downloads about 31 GB of weights into modelos/ and is the only slow one; everything after that is local. The page opens by itself at 127.0.0.1:7860, results land in salidas/, and python bootstrap.py prints what this particular machine will get — profile, resolution ceilings, expected times — before anything is installed at all.

Needed: an NVIDIA card with 8 GB or more, or an Apple Silicon Mac with 24 GB of unified memory; 40 GB of free disk; 32 GB of system RAM is comfortable. Below that it still installs, and says what it will cost.

Two ways to use it

By hand. A page opens at 127.0.0.1:7860. Twelve use cases, each showing only the inputs it needs, each opening with a worked example already in its boxes and the result of that exact example beside it. The first screen shows this + this = this instead of an empty form.

By agent. Point Claude Code, Codex or any MCP client at the folder. Twelve tools — generate, edit, describe, batch, pose library, prompt library — drive the same engine the interface drives. The model loads once and both share it, so the page can stay open while an agent works through a list of prompts.

That second door is the point. Everything the interface can do is an HTTP endpoint; the agent layer is thin on purpose. This is not a wrapper around a script, it is a local service with two front doors.

What it does

Group	Use case	What goes in
-------	----------	--------------



Text to image	New image	nothing but the prompt
	Text in the image	words in quotes come out as lettering
From a photo	New portrait	person photos
	Character in a scene	person + a photo of the setting
	Pose my character	person + pose + optional style and setting
	Transparent cutout	person → PNG with a real alpha channel
	Free	everything, nothing assumed
Edit a photo	Apply a look	image + a treatment picked from a grid
	Tell it what to change	image + a sentence + up to three references
	Match a style	image + a picture whose manner you want
	Enlarge	image, redrawn larger rather than stretched
	Replace something	image + a selection + what goes there

An edit selects its region from words or from a brush painted at the photo's real resolution. What you leave untouched comes back byte for byte — the test suite checks that rather than assuming it.

How this differs from a closed model

Not “better”. Different in ways that decide whether it fits what you are doing.

	QwenStudio, locally	A hosted closed model
Where the image goes	nowhere — no upload, no retention, no terms about your inputs	to a server, under whatever the terms say today
Cost per image	electricity	per call, forever
Works offline	yes, once the weights are down	no
The prompt you sent	exactly what you wrote, plus scaffolding you can read in a file	rewritten by a layer you cannot see
Refusals	none of its own	a moderation policy that changes without notice
Reproducibility	same seed, same weights, same image, in a year	the model is swapped underneath you



Identity from a photo	reference images, no training run	usually not offered at all
Speed	62 s at 1 MP, 239 s at 2K, on a 24 GB laptop GPU	seconds
Peak quality	very good	usually better
Licence to sell the output	no — Qwen Research License	usually yes

The honest summary: a closed model is faster, often prettier, and legally simpler to sell. This one is private, free to run, reproducible, inspectable and yours — and the prompt scaffolding that makes identity transfer work is written in a file you can open and argue with.

That last point carries the most weight. Every rule this app applies was measured rather than assumed, and the ones that turned out wrong are still written down. A closed model hands you a result. This hands you a result and the reason.

How it works

Four reference slots, one job each

The model accepts several reference images. Left to itself it blends them; the app gives each one a job and says so in the prompt, so they compose instead of fighting.

Slot	Gives	Stays out of it
Person	the face and identity	everything else
Pose	posture, limbs, head tilt	build and height — those stay the person's
Style	grade, contrast, grain, quality of light	its subject, setting and composition
Scene	place, wardrobe, framing, lighting	the face

Each reference is tagged `<image1>`, `<image2>`... and the tag is repeated inside its own instruction, because the text encoder reserves one vision slot per tag and that binding is what keeps them apart. The list handed to the pipeline and the numbering in the prompt are built in the same function, so they cannot drift apart, and every response names which slot got which tag.

Three models, one GPU

The 7B image transformer, Qwen3-VL for description and CLIPSeg for segmentation all want the same card. A small registry knows which can coexist and swaps the rest out. The image pipeline is never unmounted: with model offload its weights live in CPU memory and cost about 0.4 GB idle, so evicting it would only buy a reload. For batch work a setting keeps everything mounted and skips the swapping.



Describing an image costs no extra download, which is the neatest thing in the build: the image model's text encoder is Qwen3-VL-8B, and the checkpoint already carries the vision tower and the language head. The same 16 GB do both jobs.

Where the images go, and how they got there

Everything written lands in one folder, and the gallery reads that folder rather than a database, so the disk is the single source of truth: delete a file there and it disappears from the gallery.

The results column is not empty the first time it is opened. It carries thirty-one images this install produced, in two groups, each one clickable for the prompt, the seed and the settings behind it, and for a button that opens the use case it came from with that prompt already written. Five of them build on each other — a character invented from a paragraph, carried into a portrait, a scene and a cutout, and two of those images then edited — because one tool doing a day's work argues better than eight unrelated demonstrations. It steps aside as soon as there is work of the user's own to show.

Twelve paths and a handful of tools do not fit on one screen, so the footer carries an index of all of them, and each entry is named the way someone would ask for it rather than the way the panel is labelled. Nobody looking to remove a background guesses that it lives under "transparent cutout", so that is what the index calls it, and clicking opens that path already set up. The same footer carries the two sentences that matter before an image leaves the machine: this is a beta to experiment with, and the model's licence is non-commercial.

Each result carries its own recipe, written into the PNG as tEXt chunks: the exact prompt, the seed, the steps, the decoder, the LoRA, the reference order. Click any image and it opens, with the prompt ready to copy, to put back in the box, or the image itself ready to become the input of the next step. Into the file rather than a sidecar or a database, so it survives the file being moved, copied or sent to someone.

Looks, and what was left out

A treatment is picked from a grid rather than typed, and every thumbnail in it is that effect applied to this install's own reference photo, generated on this machine. A look can be confined to a painted region instead of covering the frame. Enlarging is its own path: the image goes back in as its own reference and is redrawn larger, which recovers detail instead of interpolating pixels. It was left out of the interface for a while because under bf16 it took 754 seconds for one image, and twelve minutes is not a feature. Quantised it takes 184 seconds, or 97 with the adapter, and the button came back when the number did. It is also the one edit path that runs without guidance, on purpose: it has no instruction to steer toward, and what guidance costs it is the size it exists to produce.

LoRAs, and a trap worth knowing

Drop .safetensors into the loras folder and a selector appears with a weight beside it. The catch is generational: the relight LoRAs circulating publicly were trained for Qwen-Image-Edit 2509 and 2511, a different architecture, and do not transfer to Qwen-Image 2.1. None is shipped as a use case here because none has been verified against this model. Loading one aimed elsewhere is treated as a user error rather than a crash. Reference-based identity already works without any LoRA, which is why there is no training step in this project at all.



Editing without touching the rest

An edit crops around the mask with padding, regenerates that crop at full resolution, and pastes it back through a feathered edge. Detail lands where the edit is instead of being spread thin over the whole frame, and the untouched area is preserved exactly.

Both editing paths run through that same routine, so a look confined to a painted region and a replacement are one operation with two prompts. The region can be named in words — CLIPSeg finds it, SAM 2 sharpens the edge — or painted by hand at the photograph's real resolution. A replacement needs a region; a look does not, and without one it grades the whole frame.

Knowing what the card is doing

The header carries a live reading of the card: memory used against memory fitted, refreshed while work runs, with a button that drops the cached allocations without unloading the model. Before each image of a batch the app can wait for the card to come back under a temperature you set. That is not protection from damage — the firmware enforces its own limits and nothing here can override them — but a long unattended run finishes sooner if it is not being throttled the whole way, and a number on screen beats guessing.

Deleting, from the gallery or from the results column, moves the file to a bin folder on disk rather than unlinking it. A grid of thumbnails that look alike is exactly where an irreversible click loses the good one; emptying the bin stays the user's decision.

Using it well

An open model takes instruction differently from a hosted one, and the differences are not guessable. Each line below was measured on the same picture with the same seed, changing one thing at a time.

Do this	Because
One photo of the person when a scene is loaded too	several are read as several different people, and several people is what comes back
Name the attribute, never the person	"replace the woman with the man" changes nothing; "replace the face, the hair and the beard" works
Prefer "put X from <image2> on..."	that construction gave the strongest exchange; "replace" and "change" graft, "edit to match" does nothing
Name the clothing when changing who someone is	the original garment anchors the original person; body and build changed nothing on their own
Put what matters last	the end of the prompt carries the most weight, which is why the app appends pose and scene after your words
Write a negative prompt before raising CFG	without one, CFG is a no-op: byte-identical output, same time



Leave reference images alone

they are encoded near 1 MP regardless; 0.26, 0.92 and 2.0 MP gave identical results

Twenty-eight steps where a person is involved, sixteen where none is. Sixteen was the default for everything once, swept on a watch movement and a hand — texture, which is resolved by sixteen. A face is not: at sixteen it comes back a thinner, younger stranger, at twenty-four close, at twenty-eight them. Resolution is capped by how much area the card can hold and by how many references are in play, not by the card alone — the app shows the ceiling before it runs, and refuses rather than reaching for it.

What we measured

All of it in one long session on an RTX 5090 Laptop with 24 GB. Where a belief turned out to be wrong, the wrong version is kept.

Reference order decides everything.

Identity transfers when the person is <image1>. With the scene first, the scene dominates and the face does not change at all — and it fails silently, producing a plausible image that is simply the wrong person. The app fixes the order: person, pose, style, scene.

A scene with a person in it will hand you that person.

The identity clause sits at the start of the prompt and the scene reference, closer to the end, outranks it. With a female scene and a male reference, the result came back as the woman from the scene: right size, right light, wrong human. The fix is recency — repeat the identity after the scene clause. Same seed, same inputs, correct face.

A reference is ignored unless the prompt names what to take from it.

With a scene loaded and a neutral prompt, the scene contributed nothing. The scaffolding clause alone was not enough. The app now describes the scene with Qwen3-VL and appends that description, which is what makes the path work at all.

"Replace X" is read as "add X on top of X".

Asking for a denim jacket over the mask of a yellow sweater returned the jacket open, with the sweater underneath. The mask had room for both and the model used it. Saying the garment is closed and that nothing shows underneath is what turns a layer into a replacement.

Prompts go positive but imperative.

At the default cfg of 1 there is no negative guidance, so "ignore the background" just injects the concept. But softening the verb breaks it too: "the face is hers" does not transfer, "the subject's face must match <image1> exactly" does.



A negative prompt does not exist below cfg 1.

The pipeline accepts a negative prompt at any cfg, and at 1 it is silently inert: same seed, byte-identical image, identical time, because the second forward pass is never run. Raised to 3 with something to push against, the same prompt turned a blurred watch movement into resolved jewels and screws for 80% more time. The control is offered only above 1, because a control that looks like it works and does not is worse than no control.

Style is a description, not a picture.

Handing the model a painting as `<image2>` and asking for its style barely moves the image, and what does come across is the painting's subject: a lighthouse landed in a portrait, and a botanical watercolour replaced the sitter with its own rosemary and fig. Naming the technique in words — flat colours, no gradients, crisp geometric edges — is what works. So the app reads the reference with Qwen3-VL first, asking only how the picture is made and never what it shows, and sends that sentence with a single image.

Steps are an identity control before they are a quality control.

At 1 MP the clock barely moves between 8 and 25 steps — 18 s to 38 s — which made steps look free. On texture the picture settles early: 8 is mush, 12 leaves a watch movement soft, 16 resolves its screws, and 20 and 25 add nothing. That sweep set the default at 16 and it was measured on the wrong subject. A face keeps changing long after a watch has stopped: same prompt, same seed, same reference, 16 returns a thinner and younger stranger, 24 gets close, 28 is the person. Four independent sources put this pipeline at 25–50, and none of them was consulted before a watch was.

There are two int8s and only one of them works here.

The first attempt used bitsandbytes' `load_in_8bit`, which is `LLM.int8()`: mixed precision with an fp16 path for outliers. Measured at 2.8 s/step and 24.1 GB peak against 1.26 s/step and 21.2 GB unquantised — slower and larger than not quantising at all, which is the opposite of the point. What the profile uses is quanto's weight-only int8: eight-bit weights, bf16 arithmetic. That one is both quicker than nf4 at the same job and better at holding a likeness, and it is what a card with twenty gigabytes or more now gets. Same two words, opposite results, which is why the recipe records which one made a picture.

Every edit ran with no guidance at all, for as long as there has been editing.

Guidance needs two halves: a scale above one and a negative prompt to steer away from. The editing paths passed a hardcoded scale and no negative prompt, so the scale did nothing — diffusers prints a warning saying exactly that, and it had been scrolling past on every edit for months. Measured on one source at one seed, 1.0, 2.5 and 4.0 return byte-identical results, and 4.0 with a negative prompt drops the speckle from 2.51 to 1.33. What it cost was not subtle: whole frames repainted in mottled concrete, and a black-and-white look that came back in colour with the effect ignored entirely. The rule was written in this document and in the README, and neither was enough to make anyone check the one path that was not following it.



An edit names the attribute, never the person.

"Replace the woman with the man from <image2>" changes nothing, twice over. "Replace the face, the hair and the beard" works. It is the rule in QwenLM's own system_prompt_edit.txt, and the failure it prevents is the one they name: under-editing, an output so close to the input that it looks like nothing ran. Two more of theirs, learned here the expensive way: the change goes before the preservation, and the preservation stays generic — listing the place, the light and the framing made the model repaint them, and what it repainted was the reference's.

The verb decides more than the documentation admits.

Same picture, same reference, same seed, the same attributes named, only the verb changed. "Put X from <image2> on ..." did the most and "swap X for ..." came close; "replace" and "change" grafted the new feature onto the old face instead of exchanging it; "give" and "edit to match" did almost nothing. Replace is the verb every example uses, this one included, and it sits in the middle.

One example is a template; two are a rule.

The prompt rewriter was given one worked example of an edit instruction and copied its wording exactly, asking for a beard on a reference that had none. Two examples differing only in what the reference showed, and it began adapting the list instead of reproducing it. The same sentence that teaches a form also teaches that the form is fixed, unless something shows it is not.

Close anything else using the GPU first.

Starting with VRAM half full makes generation crawl with no error: utilisation reads 100%, power draw stays low, nothing finishes. Half an hour went to a forgotten ComfyUI process before the app learned to warn about it.



Numbers

Text to image, 2048 px, 16 steps	232 s · 72 s with the adapter
A face, 1344 × 1792, 28 steps	278 s · 55 s with the adapter
Into a scene, two references, 1024 px	146 s
A pose or a style, two references	113 s
Masked replacement, 1344 × 1792	56 s
Editing the whole frame, 1088 × 1440	227 s, guided
Enlarging to 1536 px	184 s · 97 s with the adapter
Model load, first call	27–35 s
Segmentation, three phrases	0.9 s
Describe an image (Qwen3-VL, 4-bit)	5–9 s, 7.1 GB
Package size, without weights	22 MB
Weights	~31 GB, downloaded once

Those figures are quanto int8 on both the transformer and the text encoder, on a 24 GB card. Cards with less use nf4, which is smaller and, measured at the same job, slower — 292 seconds against 232 at 2048 px. An earlier version of this page argued that nf4 was the better mode; it was comparing against bf16 rather than against int8. The same seed in two precisions gives two different pictures, so a recipe only reproduces within one, which is why the recipe records it.

The ceiling is an area, not a side, and there is more than one of them. With no reference a 24 GB card holds 4 MP; with one it holds 2.30; with two or more, 1 MP; and editing a whole frame, which holds the picture at full size as well as producing one and runs guidance twice, holds 1.50. The profile used to store a side and square it, which is how enlarging came to ask for 3.21 MP on a path that could not have worked on any day: that side had been measured on a 3:4 frame, and a square at the same side is a third more pixels.

ComfyUI is still about four times faster for the same image: it uses int8_convrot weights with kernels built for them, a better memory manager, and prefix caching of text and reference tokens worth roughly 1.7× on edits by itself. If raw speed matters more than being self-contained, use ComfyUI. This exists to be self-contained.



How it is checked

Nineteen cases run against the live app. They check the shape of what came back, not just the absence of an exception: a 1024 square returned when 16:9 was asked for is broken even though nothing raised.

Where the instruction can be read off the image — a blazer, a kitchen, a garment that is no longer yellow — Qwen3-VL looks at the result and the case fails if the model did not do what was asked. That is what caught the layered-jacket bug: the image was the right size, the mask was right, and the edit was wrong. Shape checking alone would have passed it.

Licences

Qwen-Image 2.1	Qwen Research License — non-commercial. This is the binding one.
Qwen3-VL-8B	ships inside the same checkpoint, same licence
CLIPSeg	MIT
Inter Tight	SIL Open Font License 1.1
This code	MIT — but the model it drives is not

Nothing made with this belongs in client work without a separate licence from Alibaba. The interface borrows a palette and a typeface as a visual reference; the mark is an original symbol drawn for this repository and no trademark is used. Superside is a trademark of its owner, and this is an independent exploration with no claim of association.

The macOS path is written but has never been run. Only Windows with CUDA has been verified, and the unified-memory thresholds for Apple Silicon are judgment rather than measurement. Treat them as a starting point.